Шаг 02. Чистый код и автоматические проверки

Блок 1 · Предыдущий: 01 · Следующий: 03. Типы

> **Время:** 6–9 часов (5–8 занятий). Обоснование — TIME.md.
> Незнакомые слова — словарь.

Зачем этот шаг

В команде стиль кода не обсуждают — его проверяет робот. Настроив линтер и хуки сейчас, вы избавите себя от споров о пробелах и от половины глупых ошибок на всех следующих шагах.

Теория

- **PEP 8** — официальное соглашение о стиле кода в Python. Общий язык всего сообщества.
- **Ruff** — линтер и форматтер в одном инструменте, написан на Rust и работает мгновенно. Заменяет собой связку flake8 + isort + black и сотни плагинов.
- **Линтер против форматтера** — форматтер приводит код к единому виду: отступы, переносы, кавычки. Линтер ищет подозрительные конструкции: неиспользуемые импорты, изменяемые аргументы по умолчанию, а также имена, которые перекрывают уже существующие (например, своя переменная `list` поверх встроенного типа).
- **pre-commit** — запускает проверки автоматически перед каждым коммитом, то есть до того, как плохой код попадёт в историю. Такая автоматически запускаемая команда называется хуком.
- **Conventional Commits** — соглашение о формате сообщений коммитов, из которого автоматически собирается CHANGELOG — список изменений по версиям.
- **CI** (continuous integration) — сервер, который сам прогоняет те же проверки на каждое изменение, чтобы «у меня работает» не было аргументом.

Куда встают проверки после этого шага:



Одни и те же команды выполняются дважды: локально хуком — чтобы поймать ошибку за секунды, и в CI — чтобы её нельзя было обойти, закоммитив в обход хуков.

Что из PEP 8 встречается каждый день

Что	Как пишут	Пример
модули, функции, переменные	snake_case	`load_records`, `max_items`
классы, исключения	CapWords	`VacancySource`, `DataFormatError`
константы модуля	UPPER_SNAKE_CASE	`DEFAULT_LIMIT = 20`
внутреннее, не для внешнего использования	подчёркивание в начале	`_parse_row`

Что	Как пишут	Пример
отступ	4 пробела, без табуляции	
импорты	в начале файла, группами: стандартная библиотека, сторонние, свои	

Между функциями и классами верхнего уровня — две пустые строки, между методами — одна. Длину строки команда выбирает сама (PEP 8 предлагает 79, часто берут 88–100) и фиксирует в настройках, а не в договорённостях.

Ruff: коды правил и настройка

Каждое правило имеет код: буквы — источник правил, цифры — номер. `F` (pyflakes) — неиспользуемые импорты и неопределённые имена; `E` / `W` (pycodestyle) — стиль; `I` (isort) — порядок импортов; `B` (bugbear) — вероятные ошибки, например изменяемый аргумент по умолчанию `B006`; `UP` (pyupgrade) — устаревший синтаксис; `A` — перекрытие встроенных имён. Объяснение любого правила печатает `ruff rule B006`.

```
[tool.ruff]
line-length = 100
target-version = "py313"

[tool.ruff.lint]
select = ["E4", "E7", "E9", "F", "I", "B", "UP"]
```

Команда	Что делает
`ruff check .`	ищет нарушения и печатает их с кодом правила
`ruff check --fix .`	исправляет то, что можно исправить автоматически
`ruff format .`	переформатирует код
`ruff format --check .`	только проверяет формат; для CI

Подавление нарушения в конкретной строке — комментарий с кодом правила и причиной: `from .models import * # noqa: F403 — реэкспорт публичного API пакета`. `noqa` без кода отключает все правила в строке и прячет будущие ошибки. Форматтер Ruff сделан совместимым со стилем Black и почти всегда даёт тот же результат, поэтому второй форматтер не нужен.

Как работает pre-commit

`pre-commit install` записывает скрипт в `.git/hooks/pre-commit`, и Git вызывает его перед каждым коммитом. pre-commit запускает хуки на файлах из индекса. Если хук нашёл нарушение или сам исправил файл (форматтер, удаление пробелов), коммит отменяется: вы смотрите изменения, снова делаете `git add` и повторяете коммит.

```yaml
repos:
  - repo: https://github.com/astral-sh/ruff-pre-commit
    rev: v0.6.9                      # версии обновляет pre-commit autoupdate
    hooks:
      - id: ruff
        args: [--fix]
      - id: ruff-format
  - repo: https://github.com/pre-commit/pre-commit-hooks
    rev: v5.0.0
    hooks:
      - id: end-of-file-fixer
      - id: trailing-whitespace
      - id: check-added-large-files
      - id: check-yaml
      - id: check-toml
```

Хуки можно обойти: `git commit --no-verify` или просто не установить их после клона. Поэтому последнее слово всегда за CI, который выполняет те же проверки на сервере.

Conventional Commits

Формат первой строки — `тип(область): описание`, например `feat(cli): добавить фильтр по статусу` или `fix: учитывать часовой пояс при разборе даты`. Основные типы: `feat` — новая возможность, `fix` — исправление, `docs`, `refactor`, `test`, `ci`, `chore` — обслуживание. Восклицательный знак `feat!:` или строка `BREAKING CHANGE:` в конце сообщения отмечает несовместимое изменение. По типам инструменты сами вычисляют следующую версию (`fix` — патч, `feat` — минорная, несовместимое — мажорная) и собирают CHANGELOG.

Из чего состоит workflow GitHub Actions

Workflow — YAML-файл в `.github/workflows/`. `on` перечисляет события запуска, `jobs` — задания; каждое выполняется на свежей виртуальной машине (`runs-on`), задания по умолчанию идут параллельно. Внутри задания шаги (`steps`) идут по порядку: `uses` подключает готовое действие, `run` выполняет команду оболочки. Упавший шаг останавливает задание, и проверка на странице PR становится красной.

```yaml
name: checks
on:
  pull_request:
  push:
    branches: [main]
jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4          # получить код репозитория
      - uses: astral-sh/setup-uv@v6        # установить uv
      - run: uv sync --locked
      - run: uv run ruff check .
      - run: uv run ruff format --check .
```

Мажорные версии действий со временем растут; актуальную смотрите на странице действия. Имя задания (`lint`) потом выбирается в правилах защиты ветки как обязательная проверка.

Докстринги и PEP 20

Докстринг отвечает на вопросы «зачем» и «что вернётся», а не пересказывает код:

```
def select_open(records: list[dict], limit: int = 10) -> list[dict]:
    """Вернуть не больше limit открытых записей, самые новые первыми.

    Записи без даты создания пропускаются: их нельзя упорядочить.
    """
```

PEP 20 печатает `import this`; чаще других цитируют «явное лучше неявного», «простое лучше сложного» и «ошибки никогда не должны замалчиваться».

Материалы

- PEP 8 — читать целиком, он короткий.
- Ruff: documentation — разделы Configuration и Rules.
- pre-commit — установка и формат `.pre-commit-config.yaml`.
- Conventional Commits — на русском, страница целиком.
- PEP 20 — выберите два принципа и найдите пример в своём коде.
- GitHub Actions: Python — запуск команд в workflow.

Задание

1. Настройте Ruff в `pyproject.toml`: длина строки, целевая версия Python, набор правил. Начните с `E4`, `E7`, `E9`, `F`, `I`, затем добавляйте правила по мере понимания их смысла.
2. Прогоните линтер и форматтер по всему коду, исправьте замечания. Отключать правила можно, но каждое отключение — с комментарием, почему.
3. Настройте pre-commit: ruff, ruff-format, проверки на конец файла, пробелы в конце строк, размер файлов, валидность YAML и TOML.
4. Установите хуки локально и убедитесь, что коммит с нарушением не проходит.
5. Опишите в README или CONTRIBUTING принятые соглашения: стиль коммитов, как запускать проверки.
6. Добавьте в код докстринги к публичным функциям и классам. Докстринг — строка с описанием в начале функции; её показывает `help()` и подсказка в IDE.
7. Создайте первый workflow GitHub Actions: `pull_request` и `push` в основную ветку, checkout, установка uv, `uv sync --locked`, `uv run ruff check .` и `uv run ruff format --check .`. БД и сборка образов здесь не нужны.
8. Откройте PR с ошибкой линтера, увидьте красный CI, затем исправьте. Включите обязательность этой проверки для основной ветки. Для бесплатного учебного маршрута используйте публичный репозиторий; доступность защиты приватных репозиториев зависит от тарифа GitHub.
9. Объясните соотношение Ruff и Black. Не устанавливайте второй форматтер ради одинакового результата.

Обязательные критерии

- ☐ `ruff check .` и `ruff format --check .` проходят без ошибок.
- ☐ Настройки Ruff лежат в `pyproject.toml`, хуки — в `.pre-commit-config.yaml`, workflow — в `.github/workflows/`.
- ☐ `pre-commit run --all-files` зелёный, хуки установлены локально.

- ▢ Попытка закоммитить заведомо кривой код блокируется хуком.
- ▢ Все подавления правил снабжены комментарием с причиной.
- ▢ CI выполняет те же проверки Ruff, что и локально; ошибка проверена на PR.
- ▢ У публичных функций и классов есть докстринги.

Желательные критерии

- ▢ Слияние в основную ветку заблокировано при красной проверке (настройка репозитория).
- ▢ Соглашение о коммитах описано в репозитории и соблюдается в новых коммитах.
- ▢ Ученик объясняет два принципа PEP 20 на примерах своего кода.
- ▢ Объяснено соотношение Ruff и Black своими словами.

На что обратить внимание

- Не правьте замечания линтера вручную там, где справится `ruff format` или `ruff check --fix`.
- Форматтер и линтер не спорят друг с другом только при согласованной длине строки.
- Докстринг объясняет, зачем нужна функция и что она возвращает, а не пересказывает её код.

Вопросы для собеседования

Проверьте себя по темам этого шага. Ответов в программе нет намеренно: формулировка своими словами и есть проверка. Где можно — отвечайте на примере своего кода.

1. Чем линтер отличается от форматтера? Приведите по два примера того, что находит каждый.
2. Зачем команде единый стиль кода, если код и так работает?
3. Что делает Ruff и какую связку инструментов он заменяет?
4. Как соотносятся Ruff и Black? Нужно ли ставить оба?
5. Что такое pre-commit и зачем проверки до коммита, если есть CI?
6. Можно ли обойти pre-commit? Почему тогда нужен CI?
7. Как правильно отключить правило линтера, если оно мешает?
8. Назовите два принципа PEP 20 и приведите пример из своего кода.
9. Что такое Conventional Commits и что из них можно получить автоматически?
10. В какой момент должен запускаться CI и что он должен проверять на этом шаге?
11. Зачем включать обязательность проверок для основной ветки?
12. Что должно быть в докстринге и чего в нём быть не должно?

Полный набор по блоку — вопросы по блоку 1.